# Università degli Studi di Torino

Dipartimento di informatica


Tesi di Laurea Triennale in Informatica

# Heuristics for Team Formation in Aggregate Computing


Relatore:

**Prof. Giorgio Audrito**

Candidato:

**Federico Giacardi**



Anno Accademico 2022/2023

*Computer Science is no more about computers than astronomy is about telescopes.*

- Edsger Wybe Dijkstra

**Abstract**

The recent rise of distributed computations, in the form of IoT systems, Cyber-Physical Systems and pervasive computing has the potential to unlock value in various sectors such as wildfire monitoring, precision agriculture, civil engineering, crowd safety and edge AI. However, to make the most of all these opportunities, difficult problems must be addressed, such as decentralization, resiliency and abstraction. Aggregate Programming, a programming model based on Field Calculus, faces these issues with a layered approach, centered on the global behaviour of a group of agents, rather than on a single device. Such a shift of perspective calls for new solutions to common algorithmic problems, designed to exploit the advantages of the new paradigm fully. Hence, the goal of this work is to propose efficient heuristics for the "team formation" problem, developed by leveraging the notion of aggregate process and with particular focus on the precision of the size of the team formed, while ensuring decentralization and resiliency.

*I declare to be responsible for the content I'm presenting in order to obtain the final degree, not to have plagiarized in all or part of, the work produced by others and having cited original sources in consistent way with current plagiarism regulations and copyright. I am also aware that in case of false declaration, I could incur in law penalties and my admission to final exam could be denied*

*To my family and friends, for their support.*
*To my professors, for sharing with me their passion for research.*

# Contents


**1 Introduction** **7**
1.1 Digitalization and IoT . . . 7
1.2 The team formation problem . . . 8
1.3 Research questions and outline . . . 8

**2 Aggregate Computing** **11**
2.1 Field Calculus . . . 11
2.2 Aggregate Programming . . . 13
2.3 Aggregate Processes . . . 14
2.4 FCPP . . . 15

**3 Team Formation Problem** **17**
3.1 Introduction . . . 17
3.2 Problem definition . . . 17
3.3 Implementation . . . 20
3.4 Propagation strategies . . . 22
3.4.1 Naive Propagation . . . 23
3.4.2 Fair-share propagation . . . 23
3.4.3 Resource-based propagation . . . 24
3.4.4 Probabilistic propagation . . . 26
3.4.5 Distance-based propagation . . . 29

**4 Evaluation** **33**
4.1 Introduction . . . 33
4.2 Simulation setup . . . 33
4.2.1 FCPP setup . . . 33
4.2.2 Statistics . . . 35
4.3 Results Analysis . . . 38
4.3.1 Algorithm one . . . 39
4.3.2 Algorithm two . . . 39
4.3.3 Algorithm three . . . 41
4.3.4 Algorithm four . . . 41
4.3.5 Algorithm five . . . 42
4.3.6 Algorithm six . . . 42
4.3.7 Algorithm seven and eight . . . 42
4.3.8 Convergence speed . . . 42

**5 Conclusions** **43**

# Chapter 1

# Introduction

## 1.1 Digitalization and IoT

In recent years, access to huge computing power, connectivity and storage at a meagre cost has fueled digitalization. This process encompasses a wide range of sectors, from healthcare to e-commerce, through self-driving cars and urbanistic and it's now recognized as a strategic priority by states [9]. One of the main driving forces of digitalization has been the diffusion of mobile devices, both traditional, such as smartphones, and unconventional, like dishwashers. Furthermore, we are now witnessing the rise of wearables, like the Apple Watch. The Internet connectivity of these devices has led to a change in the structure of the Internet, from a packet-switched network of computers to a heterogeneous collection of devices, known as IoT, that can communicate with each other to add more value, e.g., by allowing the collection of a huge amount of data, that can later be processed to provide insight on business processes. This phenomenon is also reshaping computation, pushing it from a centralized, cloud-based identity, to an architecture where complexity lies at the edge, known as Edge Computing. IoT promises many impactful applications, such as, crowd monitoring for smart cities [7], robot swarms, unmanned aerial vehicles control [16], precision agriculture and wildfire monitoring [4], Edge AI. Such a solution offers some powerful advantages, including higher processing speed, and scalability. However, edge computing also poses new questions that computer science must answer, including:

- large-scale deployments;
- decentralization;
- resilience to change to nearly continual faults and changes in the network;
- opportunistic, large-scale, interactions;
- variable concentration of heterogeneous devices;
- difficult development process, given the need to specify network protocols, coordination mechanisms, and the application itself;
- device mobility;
- need for libraries and software stack.

Solving such issues requires the development of new computational models, among which we consider Aggregate Computing. This model is based on two fundamental ideas:

- program and operate distributed systems in terms of aggregates of devices rather than attempting to micro-manage each device;
- functionally composing collective behaviours to achieve effective and resilient complex behaviours in dynamic networks.

These two ideas are incarnated by Field Calculus, a functional programming model for the specification and composition of collective behaviours, above which Aggregate Computing builds, using a layered approach. This has been achieved by first considering resilience, then pragmatism in the form of reusable libraries capturing common coordination patterns, and finally applications across several different domains. This approach has already shown interesting potential in numerous fields [16], however, its full exploitation opens many research questions, such as:

- application of AP as enabling technology for Edge AI;
- deployment on microcontrollers and in limited-resource contexts;
- non-functional requirements, including security, efficiency and privacy;
- full logical formalization of the paradigm in proof assistants.

Hence, the goal of this thesis is to explore one of these questions: the development of efficient algorithms for the "team formation" problem.

## 1.2 The team formation problem

In this problem, our goal is to construct a team of devices, that can later be employed in solving a certain task. More precisely, the detection, by one of them, who will later become the leader of the team, of an external stimulus (e.g. fire alarm activation) starts the formation procedure, which can be abstracted as the propagation of a quantity, `spotsLeft`, that estimates the number of nodes that can still join the team, to its neighbours, that can decrement it by one, thus joining the team or further propagate it as it is. Such a process comes to an end when the team size has stabilized (thus implementing a "Bellman-Ford-like" termination). At the end of this procedure, the members of the team start the execution of a certain task (e.g. the exploration of a certain area to detect wildfire).

## 1.3 Research questions and outline

By reasoning on the simple model outlined before, we can identify some research questions, which will be the subject of this thesis:

1. How can we reproduce the desired behaviour in a fully decentralized way, by using Aggregate Programming?
2. How can we deal with multiple, concurrent, team formation requests?

3. How can we precisely reach the desired team size, and, at the same time, complete the creation process sufficiently fast?

This thesis is thus organized as follows. Chapter 2 provides an overview of the fundamental ideas of Aggregate Programming, required to understand the rest of the work. Chapter 3 introduces the Team Formation problem and presents some algorithms that tackle it, along with their FCPP implementation. Then, Chapter 4 evaluates the solution proposed in the previous step. Finally, Chapter 5 summarizes the work done and outlines some potential future developments.

# Chapter 2

# Aggregate Computing

## 2.1 Field Calculus

As portrayed in Figure 2.1, aggregate programming builds upon previous research threads, to elaborate a layered architecture that aims to simplify the design, creation, and maintenance of complex distributed systems. This approach is motivated by three key observations about the engineering of these systems:

- composition of modules and subsystems must be simple and transparent;
- coordination mechanism may vary across regions and time;
- mechanisms for robust coordination should be hidden by abstractions, such that programmers are not required to interact with the details of their implementation.

Among the most relevant foundations, we must cite Field Calculus (FC). FC was introduced in [18] as a minimal core calculus meant to capture the key ingredients of languages that make use of computational fields, which are functions over fields, functional composition with fields, evolution of fields over time, construction of fields of values from neighbours and restriction of a field computation to a sub-region of the network. It is therefore essential to introduce the notion of computational field [13]: this is a global, distributed map from devices to objects (data values of any sort, including higher-order objects such as functions and processes). The field calculus is based on the idea of specifying the aggregate system behaviour of a network of devices, where a dynamic neighbouring relation is used to indicate the devices with which one can directly communicate, as a functional composition of operators that manipulate (evolve, combine, restrict) computational fields. A key feature of the approach is that a specification can be interpreted either locally or globally. In the former, it can be seen as describing a computation on an individual device, iteratively executed in asynchronous "computation rounds", structured as follows:

1. sleeps for some time until it wakes up;
2. gathers information about messages received from neighbours while sleeping, in the form of fields $\phi$ mapping neighbour device identifiers $\delta$ i.e., unique numbers in an enumerable set D) to values v;
3. perceives contextual information (e.g. through sensors);

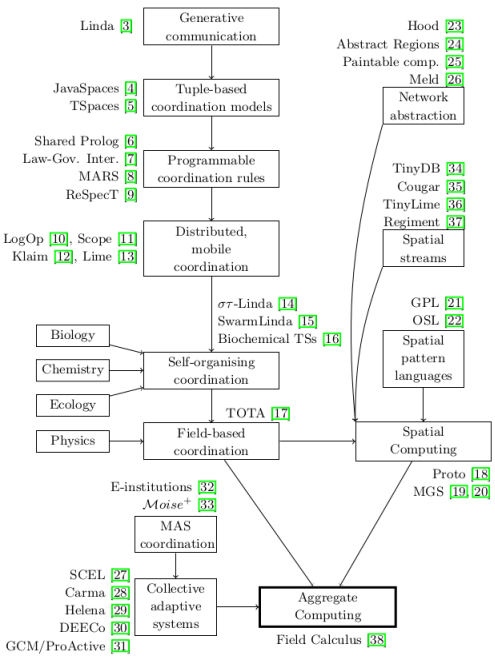


Figure 2.1: Overview of research threads leading from coordination to aggregate computing. Figure taken from [16, Fig. 2].

4. retrieves information that has been stored in the local memory of the device at the end of the previous round;

5. evaluates the program P, hence manipulating the data values received from neighbours, perceived from the context, or retrieved from the local memory;

6. finally, stores some data value (to be used in the next round) in the local store, emits a message to all neighbours with information about the outcome of computation to enable coordination, produces a value as output (that might be used, e.g., to feed actuators);

7. goes back to sleep.

Globally, a field calculus expression $e$ specifies a mapping associating each computation round of each device to the value that $e$ assumes at that space-time event. This duality intrinsically supports the reconciliation between the local behaviour of each device and the emerging global behaviour of the whole network of devices. Fields are built and manipulated using four program constructs:

- Functions: defined as $b(e_1, ...e_n)$, where $b$ is applied to arguments $e1...en$. Such "built-in" objects are stateless mathematical, logical, or algorithmic functions, sensors or actuators, or user-defined or imported library methods.

- Dynamic repetitions: structured as $rep(e_0)\{e\}$ defines a local state variable initialized with the result of $e_0$ and periodically updated with the result of executing its body statements $e$, thereby defining a field that evolves over time.

- Interaction: the expression $nbr(s)$ creates a map that associates each neighbour, as well as the executing device, to the latest value of $s$. Built-in "under the

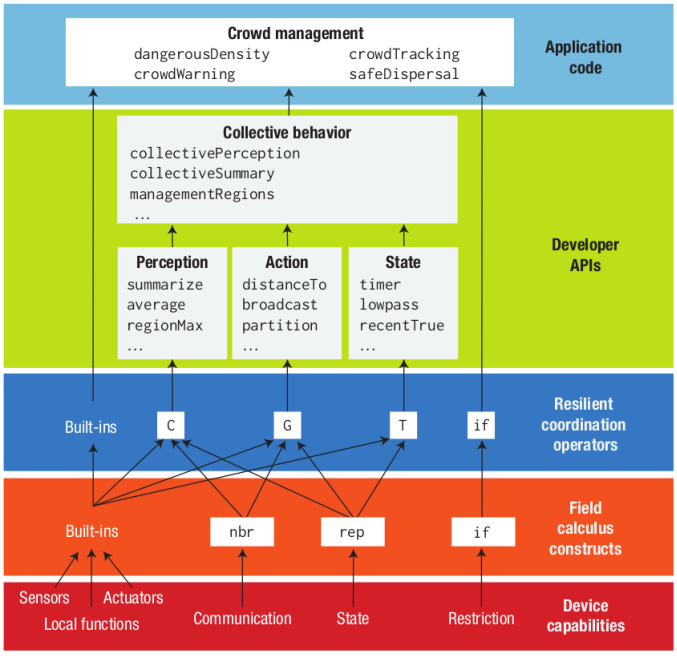


Figure 2.2: Aggregate programming abstraction layers. Figure taken from [7, Fig. 3]

hood" functions then summarize such maps: for example, $minHood(m)$ finds the minimum value in map $m$.

- Restriction: written as $if(e)\ \{e_{true}\}\ else\ \{e_{false}\}$, partitions the network into two regions: where e is true, and $e_{true}$ is computed; elsewhere, $e_{false}$ is computed instead. Importantly, partition implies that branches are encapsulated and cannot have effects outside their subspace.

The correct functioning of constructs like *nbr* and *rep* is backed by a data gathering mechanism accomplished through a process called alignment [3], which ensures appropriate message matching, i.e., that no two different instances of an *nbr* expression can inadvertently "swap" their respective messages, nor can two different instances of a *rep* expression "swap" their state memory. More formally, device state and message content are represented in a unified way as an annotated evaluation tree $\theta$. Field construction, propagation, and restriction are then supported by local evaluation "against" the collection $\theta$ of evaluation trees received from neighbours. The alignment mechanism to ensure appropriate message matching is then implemented by operations navigating these trees, and discarding them whenever different branches are taken (to prevent unwanted communication between nbr constructs in different branches of an if expression). Field Calculus is Turing universal for distributed computations as shown in [6]. However, it is too general to guarantee resilience and too mathematical and succinct in its syntax for direct programming to be simple: some methodology is needed to properly scale with complexity.

## 2.2 Aggregate Programming

Aggregate programming thus proposes two additional abstraction layers, as illustrated in Figure 2.2, for hiding the complexity of distributed coordination in com-

plex networked environments. First, the "resilient coordination operators", which consist of coordination mechanisms that are self-stabilizing, which implies that reactively adjust to changes in network structure or input values, are scalable to large networks and preserve these resilience properties when composed with one another. Any service constructed from these building blocks is thus implicitly resilient as well. One such collection of building blocks contains three generalized coordination operators plus field calculus' if and built-ins. The three operators are operator G, which is a highly general information spreading and "outward computation" operation, C, which is its inverse, a general information collection operation and T, which implements bounded state evolution and short-term memory. Implemented in field calculus, these few operators are general enough to cover, individually or in combination, many of the common coordination patterns used in large-scale systems, with strong guarantees of resilience and scalability. Above the resilience layer, aggregate programming libraries [10] [17] capture common patterns of usage and more specialised and efficient variants of resilient operators, to provide a more user-friendly interface for programming. This definition of well-organised layers of abstractions with predictable compositional semantics thus aims to foster:

- reusability, through generic components;
- productivity, through application-specific components;
- declarativity, through high-level functionality and patterns;
- flexibility, through low-level and fine-grained functions;
- efficiency, through multiple components with coherent substitution semantics [17] [15].

## 2.3 Aggregate Processes

Engineering distributed applications and services in emerging and open computing scenarios like the Internet of Things, cyber-physical systems and pervasive computing, calls for identifying proper abstractions to smoothly capture collective behaviour, adaptivity, dynamic injection and execution of concurrent distributed activities. For instance, for collaborative smartphone-based applications in a smart city, such activities may include a gossip process by which people in a plaza share comments and a guidance process to make a group of friends gather at a convenient point. This observation led to the introduction of aggregate processes, denoting a distributed computation sustained by a dynamic aggregation of devices. This notion, hence, fosters a broader view of programming smart distributed environments like sorts of distributed virtual machines for aggregate processes, supporting the creation and execution of collective computations, their diffusion over an opportunistically selected region of space-time, and their inherent self-adaptation to changes and faults by full abstraction over individual behaviours of devices. Conceptually, and technically, FC is used to specify a "single field computation" working on the entire available domain. There are mechanisms to tweak this "single field computation" model, but they are applicable only to realise an arbitrary number of computations and this practically works only if the number of such computations is small, known, and uniform across space and time. Otherwise, FC has no mechanism

to capture the abstraction of "aligned iteration" over a collection of values conceptually belonging to different computations. A second key aspect involves the ability to restrict the domain of a computation, which can't be done in FC without evaluating it outside of the area of interest. More generally, and although being universal [2], FC falls short in expressing the situation in which a field computation is composed of a set of sub-computations that is dynamic, in the sense that has variable size over space and time, which is precisely what is needed to support aggregate processes. Hence, a spawn mechanism is added to FC, essentially carrying on a multiple aligned execution of concurrent computations by managing their life cycle (i.e., activation, execution, disposal). Syntactically, this is expressed by a spawn-expression `spawn(eb, ek, ei)` modelling a collection of aggregate processes, with parameters carrying the following meaning:

- `eb` models process behaviour: it is a function, of informal type $k \rightarrow a \rightarrow (hv, bool)$, taking a process key and an input argument and returning a pair of an output value and a boolean value stating whether the process should be maintained alive or not.
- `ek` defines a field of process keysets to add at each device.
- `ei` is the input field to be fed to processes.

The result of spawn is a computational field of maps from process keys to values. As a consequence, we can precisely define an aggregate process with key $k$ as the projection to $k$ of the field of maps resulting from spawn, that is, the computational field associating each event to the value corresponding to k at that event. Generating process instances is just a matter of creating a field of keysets that becomes non-empty as soon as the proper space-time event has been recognised. Then, by `spawn`, every process instance is automatically propagated by all the participating devices to their neighbours. However, it is possible to regulate the shape of this "computational bubble" by dictating conditions by which a device must return false, which indicates the willingness to stop computing the process. That is, only devices that return status true will propagate the process. Moreover, such propagation happens continuously: so, a device that exited from a process may execute it again in the future. In particular, the border of a process bubble is given by the set of all the devices that are external but have at least one neighbour which is internal. As long as a node is in the border, it continuously re-acquires and immediately quits from the process instance: this repeated evaluation of the border is what ultimately enables a spatial extension of the process bubble (expansion). A process instance terminates when all the devices quit by returning status false. Implementing process termination may not be trivial, since proper (local or global) conditions must be defined so that the "collapsing force" can outweigh the "propagation force"; i.e., precautions should be taken so that external devices do not re-acquire the process, while the border should be steadily shrinking.

## 2.4 FCPP

Several tools have been developed during the years to support the execution of field calculus-based algorithms, among which Protelis [14] and Scafi [8]. Those tools

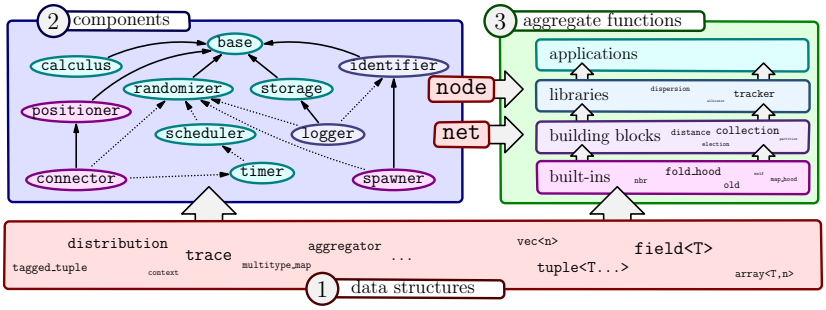


Figure 2.3: FCPP software architecture. Figure taken from [1, Fig. 1].

already cover several application scenarios, however, they both rely on the Java Virtual Machine for execution, limiting their applicability to platforms (e.g. microcontrollers) where it may not be available. Furthermore, they are not optimised for performance and do not support fine-grained parallelism, reducing their effectiveness for self-organising cloud scenarios. FCPP [1] is a more recent C++14 library implementing the Field Calculus with tools for the simulation of distributed systems, featuring:

- an extensible component-based software architecture, suitable to be customised for different application scenarios (e.g. IoT deployment, simulation, self-organising cloud) by addition of components targeting specific needed functionalities;
- a performance-oriented implementation based on compile-time optimisations, and designed to support parallel execution of a simulated system or self-organising cloud applications.

The FCPP library consists of multiple header files, divided into three main conceptual layers represented in Figure 2.3. The first layer comprises data structures needed by the second layer either for their internal implementation or for the external specification of their options, but also data structures designed for the third layer of aggregate functions: an extension of C++ arrays and tuples supporting component-wise operations, a `vec<n>` class modelling physical vectors, and, most importantly, the `field<T>` class, implementing the concept of neighbouring field (with point-wise map and fold operations). The second layer defines the abstractions for node (single devices) and net (overall network orchestration, crucial in simulations). A comprehensive list of the components currently implemented in FCPP, together with a description of their role, is given in [1]. The third layer provides the actual implementation of Field Calculus programs, as templated functions with a node parameter, and an identifier of the point where the function was called as a further parameter (needed for implementing the automatic alignment mechanism of Field Calculus for matching messages across nodes). Notice that according to the aggregate programming paradigm, aggregate functions are themselves organised in stacked layers of increasing abstraction and complexity. An aggregate function declaration consists of any number of type variables `DEF(T*)`, followed by the return type `t` and the function name `d`, followed by a parenthesized sequence of comma-separated arguments `t x` (prepended by the keyword `ARGS`), followed by an aggregate expression `e` (within brackets and keywords `CODE` return). For a more detailed presentation of FCPP, please refer to [1].

# Chapter 3

# Team Formation Problem

## 3.1 Introduction

As stated before, a full exploitation of aggregate computing to efficiently solve well-known computer science problems requires the development of new algorithms, designed to offer, among others, full scalability, fault and change tolerance and efficient resource management. This work focuses on team formation, to provide an aggregate-specific solution, capable of adjusting the size of the team formed to what is strictly necessary and minimizing the time required to form the team. Several approaches to the problem are possible: the one discussed here revolves around the idea of distributing, starting from the source, a quantity, called `spotsLeft`, that estimates how many nodes can still join the team. The peculiarities of this idea enforce a fixed structure and leave a single degree of freedom: the strategy used to propagate the aforementioned quantity from a node to its neighbours. Fine-tuning the mechanism used to compute such values led to the eight different variants presented and analyzed in the rest of this work. Hence, in this chapter, issues faced while solving the problem in aggregate computing and the common structure induced by their solution will be introduced, along with all the propagation strategies tried.

## 3.2 Problem definition

Roughly speaking, the team formation problem involves the formation, by a group of peers, in an autonomous and decentralized way, of a team of devices to solve a certain task. Formulating a proper model for such a process starts with the observation of some real-world applications, among which we should cite:

- *Robot swarms* in search and rescue missions or manufacturing, where forming a team is necessary to ensure a complete and efficient coverage of the patrolled area or assemble complex products.
- *Autonomous Vehicles*, such as self-driving cars, may need to form teams to efficiently coordinate their movements in a complex traffic system.
- *Drone Swarms*, where team formation can be used to carry out tasks like aerial mapping, and wildfire monitoring.

Abstracting away from the peculiarities of each case the following problem characteristics can be identified:

- an external stimulus requires the formation of groups of nodes, that will coordinate to solve a certain task;
- such process must be decentralized and the resulting teams must be able to perform the assigned task autonomously;
- the request might be served in real-time, thus asking for a rapid formation procedure;
- multiple formation requests could happen concurrently, which demands great accuracy from the formation algorithm.

Building on the previous ideas, a "team-state-based" model is proposed, where the formation process starts with the detection of an event by a node, that will become the team leader and leads to the initialization of `spotsLeft` to the desired team size, and of other characteristics, that depend on the considered scenario (e.g. the position the team will converge to after the formation). As stated before, `spotsLeft` models the global state of the team, as perceived by the current node, by estimating how many nodes can still join it. This quantity is then shared with neighbours using a certain strategy, to inform them of the new state of team. It's important to note that the source will generate and propagate the process only in the first round. A generic node in the team formation process then starts its round by computing `spotsLeft`, which is done by minimising the information shared by each neighbour, thus implementing, in a pair-to-pair fashion, the idea of a fully decentralized, gossip-based formation procedure. Minimization can therefore be seen as a way of taking the most updated piece of information among those shared by neighbours and reducing overfitting. According to the computed value and its state (i.e. whether it's already part of a team or not), each node can decide whether to join the current team, by decrementing the value of `spotsLeft` by one or to act as a relay agent, that just propagates traffic to its neighbours, favouring process diffusion to a wider set of nodes. The value of `spotsLeft` is then communicated to neighbours, as stated above. Such a process comes to an end when `spotsLeft` reaches zero, representing the fact that enough devices have joined the team, or when the team size has remained constant for some time. The formation phase is then followed by the execution of a given task, that depends on the considered scenario, and starts when the team size has stabilized (i.e. remained constant for a certain number of rounds). The behaviour of a generic node in a round can be summarized in the following pseudocode:

**Algorithm 1** The team-state-based model

```
function SELECT-TEAM(source, task information)
    if not source then
        spotsLeft = minimum among the spotsLeft values shared by neighbours
        if spotsLeft > 0 and spotsLeft < ∞ then
            propagate ← true
            if not yet busy then
                spotsLeft ← spotsLeft −1
                save task information
            end if
        else
            propagate ← false
        end if
    else if first round then
        propagate ← true
        spotsLeft ← desiredTeamSize - 1
        save task information
    else
        propagate ← false
        spotsLeft ← ∞
    end if
    if propagate then
        division of spotsLeft among neighbours
    end if
end function
```

## 3.3 Implementation

Trying to implement the aforementioned model in aggregate computing raises a series of issues that will now be briefly explored. In a real scenario, many events may trigger a team formation request, thus introducing the need to manage many concurrent and distributed computations, all behaving like a bubble, that spreads from the process source to its neighbours. As stated in Chapter 2, the standard tool to implement this is the aggregate process, therefore, request processing will be carried out inside the spawn callback function, which accepts, as a parameter, a tuple, containing all the team-specific information, whose members have the following meaning:

- A unique key that identifies the process. In this case, the uid of the team leader is used.
- A vector that indicates where the members of a team will converge in the simulation of the task performed after process completion. More details on this aspect will be provided in the following chapter.
- The team colour, which only serves the purpose of an easier visualization of process propagation.
- A vector that represents the position of the source node, which is used to compute distances from it in algorithms 5, 6, 7 and 8.

Such a function will return a tuple, whose first value is returned by the spawn function, while the second one represents the willingness to propagate the process to the current node's neighbours. The body of the callback function can be split into two different blocks: the first one, which retrieves information from neighbours and decides whether to join the team or not, and the second one, which shares the new state of the team with neighbours. An aggregate implementation of both blocks can be obtained through field manipulation, which shows one of the core ideas of field calculus: synchronization between nodes is implemented using the values of a field, computed using progressive transformations, in a functional fashion. Hence, the first thing to do is to construct, using the nbr function, a field that maps each node onto the value of `spotsLeft` perceived by it: its values will be used by single nodes to adjust their behaviour, effectively making it the sole tool employed to obtain the desired system global behaviour. Here, nbr does return a field, allowing the implementation of strategies that map each neighbour onto a different value. Some of the nodes that form the field may not have executed the process yet and will therefore be mapped onto `MAX_TEAM_SIZE + 1`, a finite representation of $\infty$, that represents the absence of meaningful information. The value is later updated using nbr's callback function, in which we start by retrieving by minimization, using the `min_hood` function, the value of `spotsLeft` known by every neighbour. The sole exception to this mechanism is represented by algorithm 1, where the current size, defined as `desiredTeamSize - spotsLeft` and computed by maximization, is used to take the joining and propagation decision. To understand the heuristic nature of the solution proposed in this thesis, it is important to note that a node decides whether to join a team using information coming from only its neighbours and not from the whole net. Hence, the value of `spotsLeft` computed might not be the most updated

one, and may not reflect the actual state of the team. As stated before, the value of `spotsLeft` is then partitioned appropriately among neighbours. In order to do so, a field constructed with `nbr` and containing specific information (e.g. the number of neighbours of each neighbour) is manipulated using `map_hood`. By alignment, only nodes that have already executed the same `nbr` instruction will be visible, which may lead to underestimating the real capacity of the net to satisfy the team formation request. In particular, during the first round, the source will be the only node to have executed the process and won't therefore have any information about its neighbours, which prevents the formation procedure from starting at all. To fix this, the concept of self-propagation is introduced, a mechanism by which the node currently executing the process propagates to itself the number of spots left that haven't been distributed to any neighbour (i.e. the value of `spotsLeft` after the execution of the propagation strategy). In the context of the previous problem, this means that the source will map itself onto the initial number of tokens minus one (symbolizing its participation in the team) and the process will be propagated to neighbours, that will detect, using the `min_hood` the value that the source has associated to itself, and will use it as their value of `spotsLeft`. This allows the formation procedure to start, but also increases overfitting, because the information propagated by the source to its immediate neighbours won't take into account their decision to join or not and their overall capacity to contribute to the team. As mentioned, such a mechanism is slightly more general and, crucially, can be used to deal with every situation, structural or related to the execution of `nbr`, when `spotsLeft` is greater than the number of neighbours of the current node. The simplest solution one could come up with would be to just drop the spare piece of information, but it is easy to see that doing so would not make use of all resources available on the net and may, especially for teams of bigger dimensions, lead to severe underfitting. Furthermore, nodes' mobility leaves the possibility to connect with new neighbours in the future, to which the remaining value of `spotsLeft` may be distributed. Reasoning on this problem helps identify another issue that must be solved on the way to an efficient and correct aggregate implementation: algorithms' convergence. The sole criterion for convergence introduced so far relies on the value of `spotsLeft` received from neighbours, but it's immediate to understand that not having enough free neighbours and/or the most updated information, may prevent such quantity from reaching zero, thus leading to an infinite self-propagation loop. To cope with this, a "bellman-ford" like termination criterion has been added, starting task execution, and, therefore, stopping process propagation by returning false in the spawn, when the current team size stays the same for a certain number of consecutive rounds. This mechanism is implemented using the variables `currSize`, which models the number of nodes currently part of the team and can easily be derived from the target size and the number of spots left, and `roundsAfterStabilization`, that is incremented by one each time the size of the team hasn't changed from the previous round. We conclude this section by addressing some other relevant points. First of all, it is important to note that a node may not receive valid information from any of its neighbours: in that case, `spotsLeft` will take the value `MAX_TEAM_SIZE + 1` and the process won't be further propagated. Finally, the relaying mechanism by which a node propagates a formation request even when already part of a team may lead to overfitting. To understand why, recall that a node only has access to, and would therefore propagate, information coming from its neighbours (and from itself), which

may not be updated: in this case, relaying would just serve as an error multiplier. Putting all the solutions described above leads to the following general structure, that will be adopted in all the algorithms presented in this work.

```
spawn(CALL, [&](tuple<device_t, vec<2>, packed_color,
                vec<2>> processKey){
    device_t sourceId = get<0>(processKey);
    int desiredTeamSize = node.net.storage(teamSize{});
    vec<2> teamConvPos = get<1>(processKey);
    int spotsLeft = -1;
    bool propagate = false;
    packed_color teamColor = get<2>(processKey);

    if (node.storage(performingTask{}))
        return make_tuple(true, false);

    nbr(CALL, field<int>(MAX_TEAM_SIZE + 1), [&](field<int> spotsLeftField){
        spotsLeft = min_hood(CALL, spotsLeftField);

        if (sourceId == node.uid
            && counter(CALL) == 1
        ){
            team_joining_operations();
            spotsLeft = desiredTeamSize - 1;
            propagate = true;
        } else {
            if (spotsLeft > 0 && spotsLeft < MAX_TEAM_SIZE + 1){
                if (node.uid != sourceId){
                    propagate = true;
                    if (convergencePosition == invalidPosition &&
                    !node.storage(isBusy{})){
                        team_joining_operations();
                        spotsLeft--;
                    }
                }
            }
        }

        Computation and propagation of tokens to neighbours()
    }, identifiersSet);
}
```

## 3.4 Propagation strategies

A closer look at the code presented at the end of the previous section immediately reveals that the sole point one can act on to improve performance and reach the goals stated in the introduction is the `spotsLeft` distribution logic. Hence, what follows is the presentation of several possible strategies, obtained by progressive refinement.

### 3.4.1 Naive Propagation

As stated before, each node needs to communicate to neighbours how many members are still missing from the team, hence, the simplest solution one can come up with is to map every neighbour onto the current team size, as known by the current node, using `map_hood`. If no neighbour is detected, then -1 is propagated, to symbolize the absence of meaningful information. Here it's immediate to note that only nodes that have already executed the process will be visible and associated to a value, which leads to the problem mentioned in the previous section.

```
int neighCount = sum_hood(CALL, 1);
field<device_t> neighMap = nbr(CALL, node.uid);

return map_hood([&](device_t nodeId){
    if (nodeId == node.uid && neighCount > 1)
        return -1;
    else
        return currSize;
}, neighMap);
```

Albeit simple, this solution reveals the difficulty of working in an incomplete information setting: a node has no way of knowing whether some of its peers not connected to it, that are executing concurrently (or at least after it executes the `min_hood`) will join the team. Hence, a node may perceive `spotsLeft` as not zero, and therefore join the team, even when this is not the case, which would ultimately lead to overfitting. However, this solution should be particularly fast, given that no complex (or repeated) processing of the information received from neighbours is necessary.

### 3.4.2 Fair-share propagation

To solve the problem highlighted by the previous strategy, it's important to note that only the current node has a complete list of its neighbours, even though it cannot know precisely whether they will fill some of the spots propagated by it. Hence, the approach followed from now onwards will revolve around the current node estimating how many of its neighbours will join the team and adjusting the information propagated to them accordingly. The simplest way to do so is an equal distribution of spots among neighbours, implemented as follows:

```
field<tuple<device_t, int>> neighField = nbr(
    CALL,
    make_tuple(node.uid, 1)
);
int neighboursCount = sum_hood(CALL, 1);

return map_hood([&](tuple<device_t, int> tokenLeft){
    if (neighboursCount > 0){
        return (int)(
            (nodesLeft / (double)neighboursCount) +
            node.next_real(0, 1)
```

```
        );
    } else if (node.uid == sourceKey)
        return nodesLeft;
    else
        return MAX_TEAM_SIZE + 1;
}, neighField);
```

Note that here the field manipulated for synchronization contains tuples formed by the node's id so that the source can be identified to implement self-propagation. Given the strategy considered, such a mechanism is used only when the current node has no neighbours: in all other cases, it is always possible to allocate all spots. Taking into account what has been previously said about the model, it is immediate to see that this strategy doesn't consider that different nodes may have a different number of available neighbours. For instance, node A may see `spotsLeft` = 10 and have neighbours B and C, with 7 and 1 available neighbours each. A fair share strategy would map both B and C to 5, thus leading to a team of size 7 (supposing that all nodes reached by the request decide to join the team), ultimately underfitting, in a situation where the demanded capacity could have been met.

### 3.4.3 Resource-based propagation

Building on the last observation made algorithms three and four are derived. In the former, each node is mapped onto the number of neighbours it has (or `spotsLeft`, according to what is smaller). Doing this requires an additional pass, to count how many spots won't be distributed, and should therefore be kept by the current node, which obviously increases the execution time of the algorithm.

```
int neighboursCount = sum_hood(CALL, 1);
field<tuple<device_t, int>> neighCountField = nbr(
    CALL,
    make_tuple(node.uid, neighboursCount)
);

int aux = spotsLeft;
fold_hood(CALL, [&](tuple<device_t, int> old, tuple<device_t, int> curr){
    if (spotsLeft < MAX_TEAM_SIZE + 1){
        if (get<0>(curr) != node.uid)
            aux -= std::min(spotsLeft, get<1>(curr));
    }

    return make_tuple(0, 0);
}, neighCountField);

return map_hood([&](tuple<device_t, int> currentNodeNeighCount){
    int tmp = MAX_TEAM_SIZE + 1;

    if (spotsLeft < MAX_TEAM_SIZE + 1) {
        if (get<0>(currentNodeNeighCount) == node.uid)
            return aux;
```

```
        tmp = std::min(spotsLeft, get<1>(currentNodeNeighCount));
        spotsLeft -= tmp;
    }

    return tmp;

}, neighCountField);
```

The idea applied here can be further developed by observing that only some of the neighbours will be available and, consequently, interested in joining the team. Such information can easily be incorporated by mapping devices onto 1 if and only if they haven't already joined a team (which is detected using the condition `convergencePosition == invalidPosition`, where the second variable represents the absence of a defined convergence position and, thus, not having joined a team). However, neighbours' state may change after this computation has been performed and before the arrival of the current node's request, thus (partially) invalidating the information used to propagate. This is nothing but another confirmation of the heuristic nature of the solution proposed in this work.

```
field<bool> availField = nbr(
    CALL,
    0,
    convergencePosition == invalidPosition ? 1 : 0
);
int neighboursCount = sum_hood(CALL, availField);
field<tuple<device_t, int>> neighAvailField = nbr(
    CALL,
    make_tuple(node.uid, neighboursCount)
);

int aux = spotsLeft;
fold_hood(CALL, [&](tuple<device_t, int> old, tuple<device_t, int> curr){
    if (spotsLeft < MAX_TEAM_SIZE + 1){
        if (get<0>(curr) != node.uid)
            aux -= std::min(spotsLeft, get<1>(curr));
    }

    return make_tuple(0, 0);
}, neighAvailField);

return map_hood([&](tuple<device_t, int> currentNodeInfo){
    int tmp = MAX_TEAM_SIZE + 1;

    if (spotsLeft < MAX_TEAM_SIZE + 1){
        if (get<0>(currentNodeInfo) == node.uid)
            return aux;

        tmp = std::min(spotsLeft, get<1>(currentNodeInfo));
```

```
        spotsLeft -= tmp;
    }

    return tmp;
}, neighAvailField);
```

### 3.4.4 Probabilistic propagation

As mentioned before, communication between nodes may fail, due to many external factors. In that case, the part of the value of `spotsLeft` that has been assigned to that node will be lost, and totally ignoring such eventuality would certainly lead to under-fitting. However, including it in a model is not an easy task, given the number of factors that influence communication. In this context, we can use the abstraction provided by FCPP, where neighbouring is a probabilistic relation with a communication successes distribution defined in [11] and implemented in this work as

```
FUN real_t commSuccessProbability(ARGS, vec<2> pos1, vec<2> pos2){
    real_t m_r50 = halfRad / 100;
    real_t m_k = log(6792093.0/29701) / (1-m_r50);
    real_t r99 = 100;
    return 1/(7 * exp((m_r50 - norm(pos1 - pos2)/r99) * m_k) + 1);
}
```

More details on this aspect will be given in the following chapter. Algorithm five is then designed starting from the fourth and then scaling `spotsLeft` by the expectation of the number of nodes that might be reached passing through the considered neighbour(i.e. the number of spots propagated multiplied by the probability of a successful communication).

```
field<bool> availField = nbr(
    CALL,
    convergencePosition == invalidPosition ? 1 : 0
);
int neighboursCount = sum_hood(CALL, availField);
field<tuple<device_t, vec<2>, int>> posNeighAvailField = nbr(
    CALL,
    make_tuple(node.uid, node.position(), neighboursCount)
);

int aux = spotsLeft;
fold_hood(CALL, [&](tuple<device_t, vec<2>, int> old,
                    tuple<device_t, vec<2>, int> curr){
    int tmp = MAX_TEAM_SIZE + 1;

    if (spotsLeft < MAX_TEAM_SIZE + 1){
        if (get<0>(curr) != sourceId){
            tmp = std::min(spotsLeft, get<2>(curr));
            aux -= (int)std::round((tmp *
                                    commSuccessProbability(
```

```
                                        CALL,
                                        sourcePosition,
                                        get<1>(curr)
                                    )
                                )
                            );
        }
    }

    return make_tuple(0, invalidPosition, 0);
}, posNeighAvailField);

return map_hood([&](tuple<device_t, vec<2>, int> currNodeInfo){
    int tmp = MAX_TEAM_SIZE + 1;

    if (spotsLeft < MAX_TEAM_SIZE + 1){
        if (get<0>(currNodeInfo) == node.uid)
            return aux;

        tmp = std::min(spotsLeft, get<2>(currNodeInfo));
        spotsLeft -= (int)std::round((tmp *
                                    commSuccessProbability(
                                        CALL,
                                        node.position(),
                                        get<1>(currNodeInfo)
                                    )
                                )
                            );
    }

    return tmp;
}, posNeighAvailField);
```

Algorithm six follows the same philosophy but is derived from algorithm two, as it fairly splits tokens among available neighbours.

```
field<bool> availField = nbr(
    CALL,
    convergencePosition == invalidPosition ? 1 : 0
);
int neighboursCount = sum_hood(CALL, availField);
field<tuple<device_t, vec<2>>> posNeighAvailField = nbr(
    CALL,
    make_tuple(node.uid, node.position())
);

int aux = spotsLeft;
fold_hood(CALL,
    [&](tuple<device_t, vec<2>> old, tuple<device_t, vec<2>> curr){
```

```
            int tmp = MAX_TEAM_SIZE + 1;

            if (spotsLeft < MAX_TEAM_SIZE + 1){
                if (get<0>(curr) != sourceId){
                    tmp = (int)((spotsLeft / (double)neighboursCount)
                        + node.next_real(0, 1)
                    );
                    aux -= (int)std::round((tmp * commSuccessProbability(
                        CALL,
                        sourcePosition,
                        get<1>(curr)))
                    );
                }
            }

            return make_tuple(0, invalidPosition);
        },
    posNeighAvailField);

    return map_hood([&](tuple<device_t, vec<2>> currNodeInfo){
        int tmp = MAX_TEAM_SIZE + 1;

        if (spotsLeft < MAX_TEAM_SIZE + 1){
            if (neighboursCount > 0){
                tmp = (int)((spotsLeft / (double)neighboursCount)
                    + node.next_real(0, 1)
                );
                return (int)std::round((tmp * commSuccessProbability(
                    CALL,
                    sourcePosition,
                    get<1>(currNodeInfo)))
                );
            } else if (node.uid == sourceId)
                return aux;
        }

        return tmp;
    }, posNeighAvailField);
```

It is important to note that there is no guarantee that the random number added to the number of spots propagated will be the same in both cycles. This introduces a discrepancy between the number of spots that a node will ask itself, computed in the first `fold_hood` and that of spots effectively left unassigned after the second `fold_hood`, which may result in either underfitting or overfitting, depending on generated values. The impact of this error might not be negligible, in particular for big teams, and it might be amplified during propagation. A possible solution to this problem could be obtained by declaring `posNeighAvailField` as a field of `tuple<device_t, real_t, vec<2>>`, where the second element contains the number of spots propagated to each neighbour, to be computed during the `fold_hood`

and assigned in the `map_hood`.

### 3.4.5 Distance-based propagation

Developing previous algorithms has posed a new question: which nodes should be reached by a team formation request? So far, no restrictions have been posed and propagation happened in all directions. However, it makes sense to formulate the problem as a request propagating through a distance gradient, thus reaching only neighbours of a node that are more distant than it from the source. It is reasonable to expect that such a solution would result in fewer nodes being involved in the problem, thus reducing overfitting coming from being forced to operate with incomplete information. Moreover, not propagating backwards reduces the risk of sending a request to a node that doesn't have the most updated value of `spotsLeft`. This idea is the basis of the seventh and eighth algorithms, which are obtained from the third and the fourth, respectively, by augmenting the information sent by a node to its neighbours with its distance from the source, computed using the library function `abf_distance`. Each neighbour is therefore mapped onto a valid value if and only its distance from the source, contained in the same tuple with the number of (available) neighbours, is greater than that of the current node.

```
int neighboursCount = sum_hood(CALL, 1);
real_t distFromSource = abf_distance(CALL, node.uid == sourceId);
field<tuple<device_t, real_t, int>> neighPosCountField = nbr(
    CALL,
    make_tuple(node.uid,
               distFromSource,
               neighboursCount
        )
);

int aux = spotsLeft;
fold_hood(CALL, [&](tuple<device_t, real_t, int> old,
                    tuple<device_t, real_t, int> curr){
    int tmp = MAX_TEAM_SIZE + 1;

    if (spotsLeft < MAX_TEAM_SIZE + 1){
        if (get<1>(curr) > distFromSource) {
            aux -= std::min(spotsLeft, get<2>(curr));
        }
    }

    return make_tuple(0, 0.0, 0);
}, neighPosCountField);

return map_hood([&](tuple<device_t, real_t, int> currNodeInfo){
    int tmp = MAX_TEAM_SIZE + 1;

    if (spotsLeft < MAX_TEAM_SIZE + 1){
        if (get<0>(currNodeInfo) == node.uid)
```

```
            return aux;

        if (get<1>(currNodeInfo) > distFromSource){
            tmp = std::min(spotsLeft, get<2>(currNodeInfo));
            spotsLeft -= tmp;
        }
    }

    return tmp;
}, neighPosCountField);

field<bool> availField = nbr(CALL,
                            0,
                            convergencePosition == invalidPosition ? 1 : 0
                        );
int neighboursCount = sum_hood(CALL, availField);
real_t distFromSource = abf_distance(CALL, node.uid == sourceId);
field<tuple<device_t, real_t, int>> neighPosCountField = nbr(CALL,
                                        make_tuple(node.uid,
                                        distFromSource,
                                        neighboursCount)
                                    );

int aux = spotsLeft;
fold_hood(CALL, [&](tuple<device_t, real_t, int> old,
                    tuple<device_t, real_t, int> curr){
    int tmp = MAX_TEAM_SIZE + 1;

    if (spotsLeft < MAX_TEAM_SIZE + 1){
        if (get<1>(curr) > distFromSource && get<2>(curr))
            aux -= std::min(spotsLeft, get<2>(curr));
    }

    return make_tuple(0, 0.0, 0);
}, neighPosCountField);

return map_hood([&](tuple<device_t, real_t, int> currNodeInfo){
    int tmp = MAX_TEAM_SIZE + 1;

    if (spotsLeft < MAX_TEAM_SIZE + 1){
        if (get<0>(currNodeInfo) == node.uid)
            return aux;

        if (get<1>(currNodeInfo) > distFromSource
            && get<2>(currNodeInfo)
        ){
            tmp = std::min(spotsLeft, get<2>(currNodeInfo));
            spotsLeft -= tmp;
        }
```

```
        }

        return tmp;
    }, neighPosCountField);
```

# Chapter 4

# Evaluation

## 4.1 Introduction

Having described and provided many possible implementations of algorithms for team formation in aggregate programming, the next natural step is evaluating their performances. Such activity will be carried out with two goals:

- Understand the pros and cons of each algorithm to efficiently deploy them and suggest possible improvements;
- Asses the correctness of the implementation proposed, to see whether the theoretical analysis done in the previous section is backed by empirical evidence.

Doing so implies setting up an expressive digital simulation environment, as well as the definition of insightful statistics. Hence, this chapter will begin with the introduction of tools used, along with an analysis of the measures constructed and their meaning. Finally, plots produced by the execution of algorithms in this environment will be analyzed.

## 4.2 Simulation setup

### 4.2.1 FCPP setup

Realizing a meaningful simulation scenario starts with a closer look at the possible use cases presented in the previous chapter. Doing so highlights some issues that must be dealt with in real-world contexts, that will now be discussed. First of all, some nodes may not be able to join the team due to external reasons (e.g. being involved in a task that requires all their resources). To account for this aspect, a parameter, called `nodesBusyPercentage`, has been added, containing the percentage of nodes that will be considered busy due to external reasons and therefore unable to join any team. These nodes will, however, be able to act as relay agents if necessary. Another parameter, `isBusy`, is necessary, to represent the state of the single node, with possible values 0, symbolizing a free node and 1, symbolizing a busy one. The value of this variable is

```
node.storage(isBusy{}) = constant(CALL, node.next_real(0, 100))
< node.net.storage(nodesBusyPercentage{});
```

effectively interpreting `nodesBusyPercentage` as a probability. Here, the constant function ([11]) is deployed, to make sure that the same value will be extracted in all rounds, thus ensuring that the initial configuration won't change. Secondly, nodes may be moving even when they haven't joined a team. To model this aspect, the library function `rectangle_walk` is used, to make nodes move in a rectangle at a given speed, set through the `randomWalkSpeed` parameter because it is expected to have a considerable impact on the result, given that it influences node mobility. As anticipated in the previous sections, communication with nodes, and, consequently, the neighbouring relationship, may be unstable due to external factors (e.g. electromagnetic interference), which can be incorporated into this model using the `connect::radial` construct offered by FCPP([11]). The results discussed below have been obtained using the following value

```
connector<connect::radial<
        fcpp::coordination::halfRad,
        connect::fixed<100, 1, dim>
    >
>
```

Abstractly speaking, each node is regarded as the centre of a circumference of radius 100, and its neighbours are all those falling inside this area. The probability of the communication with a neighbour being successful thus depends on the distance of the latter from the source. Readers interested in the resulting probability distribution should consult [11]. To obtain a more realistic model, communication instability has to be (partially) compensated by storing messages for a longer period. This can be done with the option `retain<metric::retain<20,1>>`, which instructs nodes to keep messages in memory for 20 seconds before dropping them. In all simulations performed, a fixed source has been used. Different nodes may have access to a more or less numerous set of free neighbours, but this has limited impact on the algorithms' result (it only marginally affects convergence times) thanks to the relay system, by which free nodes outside of the initial neighbourhood can be reached, hence, the decision not to include a parameter to regulate this aspect. Simulations have been carried out employing both the batch and the interactive mode of FCPP, along with its GUI ([5]). The latter has mainly been used to "visually" validate what was observed during the theoretical analysis and ensure that the simulation scenario was behaving as expected, while the former has been employed to automatically simulate multiple executions with different parameters' configurations, from which data analyzed in the rest of the chapter were obtained. As mentioned before, each team is formed to execute a certain task: albeit not central, this is an aspect that is included in our simulation, through the following code

```
FUN vec<2> performTask(ARGS,
    vec<2> convergencePosition,
    int * roundElapsed,
    int currSize,
    int oldCurrSize
)
{
    using namespace tags;
    bool condx = false;
```

```
    bool condy = false;
    real_t diameter = constant(CALL, node.next_real(0, 7));

    if (currSize == oldCurrSize ||
        node.storage(roundsAfterStabilization{}) == stabDelay
    ){
        if (node.storage(roundsAfterStabilization{}) != stabDelay)
            node.storage(roundsAfterStabilization{}) =
            node.storage(roundsAfterStabilization{}) + 1;

        if (convergencePosition != invalidPosition &&
            node.storage(roundsAfterStabilization{}) == stabDelay
        )
        {
            node.storage(performingTask{}) = 1;
            node.velocity() = unit(convergencePosition - node.position()) * 10;
            condx = std::abs(node.position()[0] - convergencePosition[0]) <= 5.0;
            condy = std::abs(node.position()[1] - convergencePosition[1]) <= 5.0;
            if (condx && condy)
                node.velocity() = make_vec(0, 0);
        }
    }

    return convergencePosition;
}
```

which defines a simple motion towards the position specified by the team leader. Such movement starts after the elapsing of the team size stabilization delay, controlled by the constant `stabDelay`, which has been set to 20 in our simulation, and finishes when the indicated point is reached (note that here proximity is tested, because a round may never fire when the node is exactly in the desired position). While performing the task, nodes keep a cruise speed defined as multiple of the distance between their current position and the target one. It is important to note that the execution of the task is external to the process. In the future, this aspect might be changed to perform it directly inside the spawn callback, thus obtaining a more flexible solution.

### 4.2.2 Statistics

Measuring the quality of the algorithms proposed requires the definition of expressive statistics, along with their implementation in FCPP, and a visualization of the results. The first significant measure considered is the $currentToDesiredRatio$, which is a function of type $rounds \to \mathbb{R}$ defined as $currentToDesiredRatio(r) = \frac{current_team_size}{teamSize}$, where the denominator, given as a parameter of the simulation, represents the size the team should have, while the numerator, which models the effective size, is defined as follows:

$$\sum_{n \in nodes} joined(r, n)$$

where:

$$joined(r, n) = \begin{cases} 1, & \text{if node n belongs to a team in round r} \\ 0, & \text{otherwise} \end{cases} \tag{4.1}$$

and it is implemented as a node environment variable. This measure must be interpreted as an indicator of both the speed, at which the process is propagating, and of the precision of the formation algorithm, defined as the difference between desired and current size. The ideal behaviour of this quantity is to rapidly converge to 1, which means that a team with exactly the desired size has been constructed in a small amount of time. Values over 1 mean that the algorithm is overfitting, while values lower than 1 indicate an underfitting situation. This is the most significant statistic among those considered here and it has been studied both as:

$$\lim_{round \to \infty} currentToDesiredRatio$$

to analyze the precision of each algorithm for different values of $teamSize$, and as a function of time, to analyze the transitory period. To measure the precision of the formation algorithm for different values of $teamSize$, to be intended as its ability to meet the requested size without adding useless members, the $meanError$ statistic is introduced, defined as:

$$meanError = \lim_{round \to \infty} |teamSize - \sum_{n \in nodes} joined(round, n)|$$

The monotonicity of this function is of particular interest: growth or stationarity reveals that the number of nodes added to the team increases as $teamSize$ does, while a decrease means the opposite, and it's an indicator of the algorithm's difficulty in stopping when $spotsLeft$ reaches zero. Finally, a statistic to measure speed for different values of $teamSize$ is introduced. $meanRoundsTaken$, defined as:

$$\lim_{r \to \infty} \frac{\sum_{n \in nodes} roundsTaken(r, n)}{\sum_{n \in nodes} joined(r, n)}$$

where $roundsTaken$ is a function, implemented by a node environment variable, that counts the number of rounds elapsed before the node joined a team, and therefore measures the speed of the formation algorithm. The interpretation is, again, trivial: the lower the value, the faster the formation process will be. These statistics must be discretized to be implemented in FCPP, which requires the introduction of the concepts of aggregators and functors. Information needed to compute the desired statistics is stored in both the network-wide memory, called `net_store`, and shared between all nodes, and the `node_store`, local to the single node. Data stored in such memories are then collected, using an aggregator (e.g. `aggregator::sum`), and composed, implementing the functions described above, to derive quantities to be plotted using a functor (e.g. `functor::div`). This is done at each log event, whose sequence is determined by the value of the `log_schedule` option, which, in our simulations, was set to `sequence::periodic_n<1, 0, 1, 30>`. The list of aggregators and functors used in this thesis is defined as follows:

```
using aggregator_t = aggregators<
    node_size,                  aggregator::mean<double>,
    joined,                     aggregator::sum<double>,
```

```
    roundsTaken,                aggregator::sum<int>,
    node_currSize,              aggregator::sum<int>
>;

using statsFunctors = log_functors<
    currentToDesiredRatio, functor::div<
    fcpp::aggregator::sum<fcpp::coordination::tags::joined, 2>,
    fcpp::coordination::tags::teamSize
>,
meanError, functor::abs<functor::sub<
    fcpp::coordination::tags::teamSize,
    fcpp::aggregator::sum<fcpp::coordination::tags::joined, 2>
>>,
meanRoundsTaken, functor::div<
    fcpp::aggregator::sum<fcpp::coordination::tags::roundsTaken, 2>,
    fcpp::aggregator::sum<fcpp::coordination::tags::joined, 2>
>>;
```

Plots were generated via Asymptote, a powerful descriptive vector graphics language ([12]). FCPP offers a `plot_t` object, which can be passed as a parameter of both interactive and batch simulation, so that values, computed as stated before, can be stored in it. At the end of the simulation, the following code can be executed:

```
std::cout << plot::file("teams", p.build())
```

which exports data in a .asy file, that is later fed to an Asymptote script, which effectively generates plots in a single PDF file. The structure of the latter is dictated by the value of the `plot_type` option, defined as follows:

```
    using plot_t = plot::join<
            plot::split<
                common::type_sequence<teamSize,
                        randomWalkSpeed,
                        nodesBusyPercentage
                >,
                plot::join<
                    plot::split<
                        plot::time,
                        plot::split<common::type_sequence<algoNumber>,
                            plot::value<currentToDesiredRatio,
                                aggregator::max<double>
                            >
                        >
                    >
                >
            >,
            plot::split<
                common::type_sequence<randomWalkSpeed,
                    nodesBusyPercentage
                >,
```

```
                plot::join<
                    plot::filter<
                        plot::time,
                        filter::above<30>,
                        plot::split<
                            fcpp::coordination::tags::teamSize,
                            plot::split<
                                common::type_sequence<algoNumber>,
                                plot::value<currentToDesiredRatio,
                                    aggregator::max<double>
                                >
                            >
                        >
                     >,
                    plot::filter<
                        plot::time,
                        filter::above<30>,
                        plot::split<
                            fcpp::coordination::tags::teamSize,
                            plot::split<
                                common::type_sequence<algoNumber>,
                                plot::value<meanRoundsTaken,
                                    aggregator::max<double>
                                >
                            >
                        >
                    >,
                    plot::split<fcpp::coordination::tags::teamSize,
                        plot::split<common::type_sequence<algoNumber>,
                        plot::value<meanError, aggregator::mean<int>>>
                    >
                >
            >
        >;
```

This code creates a file with two rows, one with plots that model the evolution in time of `currentToDesiredRatio`, for each assignment of values to `teamSize`, `randomWalkSpeed` and `nodesBusyPercentage`. The second one contains, for each assignment of values to `randomWalkSpeed` and `nodesBusyPercentage`, the evolution for `teamSize` of the values `currentToDesiredRatio` and `meanRoundsTaken` converge to (i.e. values at the end of the simulation) and `meanError`. It's important to note that the limit is implemented by filtering values above the final instant of simulation, using the `filter::above<30>` option.

## 4.3 Results Analysis

Having described the tools and the statistics used, it is now possible to undergo a detailed analysis of the plots generated by simulations carried out in blocks of 3 with

the following values:

- `teamSize` values that range between 5 and 50, and grow by 5 at each round;
- `randomWalkSpeed` set to 2;
- `nodesBusyPercentage` set to 30.

The following analysis will follow the same steps used to derive algorithms so that each difference in performance can be linked to structural reasons.

First of all, the behaviour of each algorithm can be decomposed in a growing phase, where team size increases to reach the desired value, and a plateau phase, where size remains constant, indicating that either the desired size has been reached or there are no additional nodes available, which is consistent with what expected. Furthermore, these two phases have a length that varies between algorithms, thus backing the hypothesis on the predominance of the propagation strategy in determining the speed and the precision of each algorithm.

### 4.3.1 Algorithm one

Starting from algorithm one, a massive overfitting tendency, that gets worse as team size increases, can be noticed, which is perfectly consistent with each node not knowing how many of its neighbours will join the team, as underlined during the theoretical analysis. It must be noted, as well, that this algorithm's transitory lasts longer than that of other algorithms, which can be attributed to the higher number of tokens being inserted into the net, which translates into more devices being involved in the process. Hence, the algorithm is not slower than the others, consistently with the particularly simple propagation strategy applied. Furthermore, this strategy, but the same behaviour is shown also by the seventh and the eighth, has a `currentToDesiredRatio` and a *meanError* that increases up to a certain value of team size, but decreases for bigger values. This can be attributed to the fact that the higher the number of nodes involved in the process, the more difficult it is to keep the value of `spotsLeft` known to each of them aligned, resulting in a scarce sensitivity to this parameter, which, in turn, means being unable to correctly detect when it reaches zero and no more members should be added.

### 4.3.2 Algorithm two

Turning to algorithm two, the opposite behaviour can be noticed. In this case, *currentToDesiredRatio* decreases as time size increases, but the *meanError* increases and has no flex points, indicating that the current size never exceeds the desired size. As predicted when explaining the propagation strategy, not considering each node's real capacity leads to unbalanced requests, and now experimental data seem to suggest that the macro effect of these errors is an underestimation of the network's real capacity, which translates into teams significantly smaller than desired. Here, it's worth observing again that the apparent speed of the algorithm comes indeed from the small number involved in the process, which means fewer contributions (especially big ones, coming from nodes joining near the end of the process) to the summation.

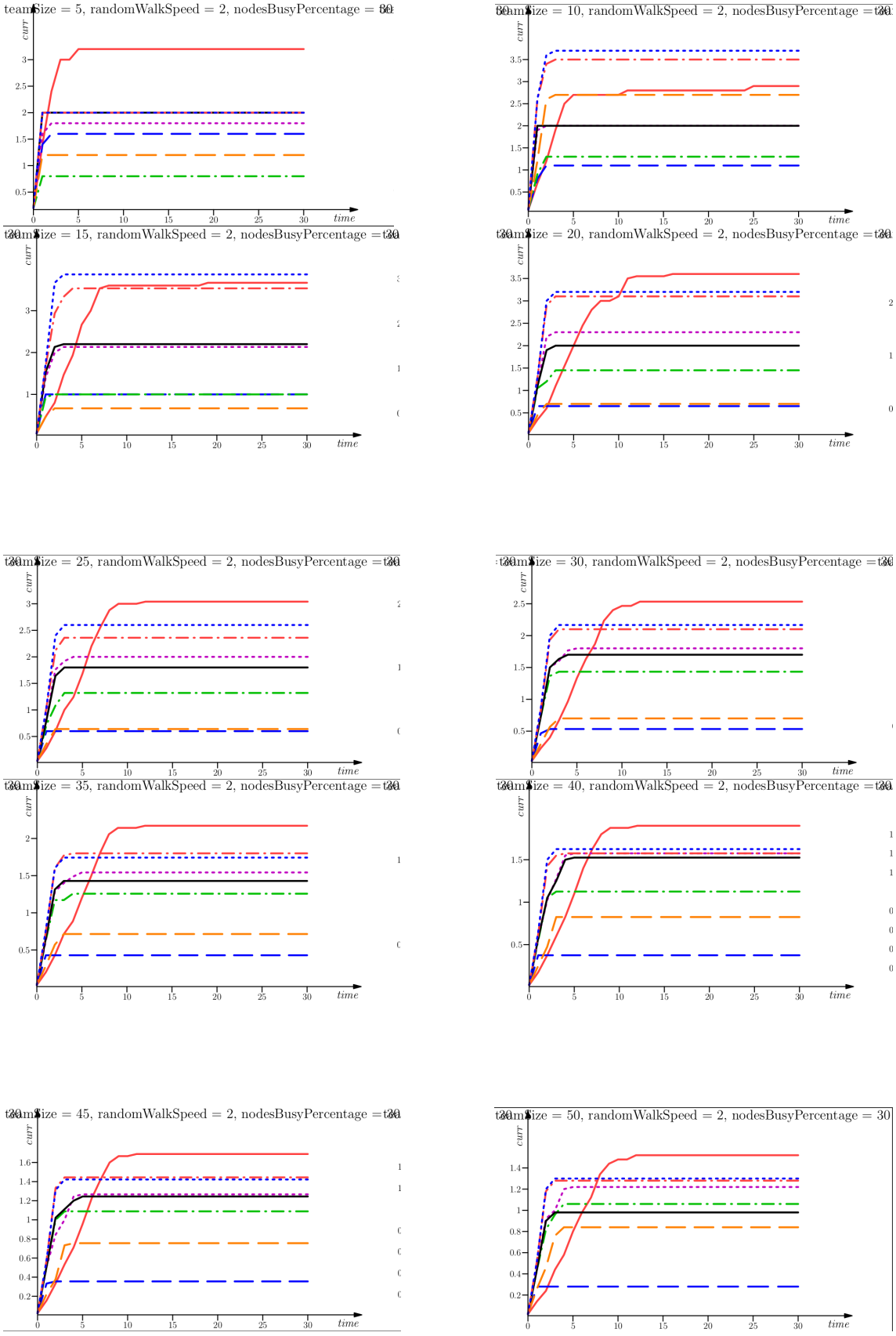


Figure 4.1: Evolution of current to the desired ratio during team formation

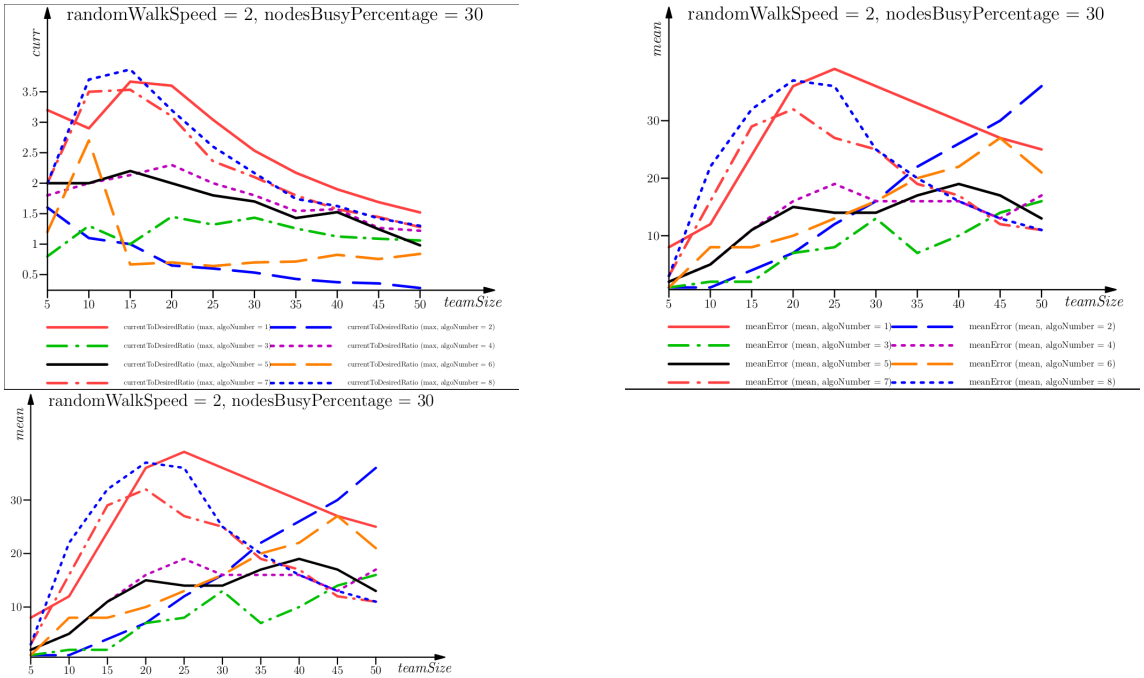


Figure 4.2: Team size dependent quantities

### 4.3.3 Algorithm three

Data also suggest that algorithm three is among the most precise and stable, with a *currentToDesiredRatio* that never exceeds 1.5, which confirms that taking into account the number of neighbours of a node leads to better results. Crucially, in this case, *currentToDesiredRatio* doesn't show the decreasing behaviour present in other algorithms, which suggests a higher sensitivity to the current size of the team. This hypothesis is confirmed by the oscillating behaviour of *meanError*. This, combined with the good propagation speed, and the fact that *meanError* is the smallest among all algorithms, suggests that this propagation strategy represents the best balance between speed, simplicity and precision.

### 4.3.4 Algorithm four

When analyzing algorithm four, the first discrepancy with the theory introduced in the previous chapter can be found. Albeit similar, both in terms of error made and convergence speed, as expected, to the previous one, this strategy has a bigger *currentToDesiredRatio*. In contrast, the opposite would have been natural, given the stricter propagation criterion, which should give a better estimate of a node's real capacity. This reveals a reduced sensitivity, as confirmed by the monotonic decreasing behaviour of the *currentToDesiredRatio*, to the current team size, which can be explained by noticing that notifying fewer spots to each node means propagating a valid number of spots to more nodes, which increases process spreading. As stated above, this makes synchronization between the various versions of the information more difficult. However, here the values of *currentToDesiredRatio* are lower, and the *meanError* shows an oscillating behaviour, indicating that progresses have been made if compared to the first algorithm.

### 4.3.5 Algorithm five

Algorithm five shows very similar behaviour to algorithm four, which is consistent with the former being the probabilistic version of the latter. Values of $currentToDesiredRatio$ and $meanError$ are actually slightly lower than for algorithm four, which is actually the opposite of what was expected because compensating for communication faults should lead to fewer values of `spotsLeft` being lost and more nodes being added to the team. In this case, switching to a probabilistic strategy didn't significantly affect convergence time, which is consistent with the low computational impact of random numbers generation.

### 4.3.6 Algorithm six

Algorithm six behaves similarly to algorithm two, which is consistent with its definition. However, a reduction of the underfitting tendency can be noticed, confirming that a probabilistic strategy successfully compensates for communication faults. This hypothesis is further backed by the reduction in the $meanError$ value. Furthermore, data suggest a higher sensibility to the value of `spotsLeft` being propagated, with the effective size constantly increasing as the desired one grows. The value registered for $teamSize = 10$ can probably be attributed to the difference between numbers computed in the two `fold_hood`, as discussed in the previous section. Speed-wise, the higher values registered here are justified by the same argument used while explaining algorithm two.

### 4.3.7 Algorithm seven and eight

Algorithms 7 and 8 show again a counter-intuitive behaviour, with $currentToDesiredRatio$ increasing even if a stricter propagation strategy is adopted. This, along with the behaviour of $meanError$, which is the same as described for algorithm one, strengthens the argument formulated about the difficulty of aligning information among an increasing number of nodes. The differences between their values of $currentToDesiredRatio$ are similar to those between algorithms 3 and 4, from which these algorithms are derived. However, it must be observed that $currentToDesiredRatio$ and $meanError$, despite showing the same behaviour, reach lower values, confirming that algorithm three constitutes a better base version than the first one. This reasoning is also supported by the convergence time of these two algorithms, which is significantly lower than that of the first and aligned with the rest.

### 4.3.8 Convergence speed

Lastly, differences in speed between algorithms 3 to 8, can mainly be attributed to the different number of nodes involved in the process, and, only in the case of algorithms 1 and 2 versus the rest, also partially due to the intrinsic complexity of the propagation strategy. This is consistent with strategies from 3 to 8 being slower, as underlined during the theoretical analysis, then the first and the second, but similar between each other.

# Chapter 5

# Conclusions

In this work, several algorithms for the resolution of the team formation problem have been introduced. Their goal is to form, in a distributed and autonomous way, a group of nodes that will later coordinate to execute a given task. The aggregate programming paradigm has been employed, in particular its notion of aggregate process, leveraging its expressiveness in modelling distributed computations. A model of this process has been elaborated, revolving around the idea of a quantity, `spotsLeft`, representing spots in the team that still have to be filled and distributed by the source to other nodes of the net. The characteristics of such a model impose a fixed structure, with a single degree of freedom, represented by the strategy used to distribute `spotsLeft` among neighbours. In the first two algorithms, strategies based on an equal distribution of spots between neighbours are employed, with experimental data indicating severe overfitting (algorithm one) and underfitting (algorithm two), due to misalignment in the value of `spotsLeft` known to each node and an imprecise estimate of the number of contributions each node can offer (i.e. nodes reachable passing through it). To solve these problems, three different ideas are introduced:

1. propagating according to the number of free neighbours, to better estimate nodes' capacity and reduce overfitting;
2. compensating communication faults using a probabilistic distribution;
3. limiting propagation to sole nodes that are more distant from the source, to reduce overfitting.

Data collected during simulations only partially support the theoretical analysis, in particular, ideas one and three lead to an increase in the number of nodes involved in the process, which worsens the misalignment problem mentioned above, thus increasing overfitting. Findings about the second point are not conclusive, with algorithm six registering an improvement, while algorithm five behaves the opposite of what was expected. Given the data at our disposal, algorithm three seems the best starting point, with possible improvements coming from probabilistic propagation and, most importantly, from misalignment limitation. Starting from the problems outlined above, several future research directions can be proposed.

- A probabilistic version of algorithm three should be implemented to obtain more conclusive evidence on probability as a way to compensate for communication faults.

- Additional work should be done to reduce information misalignment and effectively stop process propagation when `spotsLeft` reaches zero. Given that incomplete information is an intrinsic characteristic of the model used here, this probably implies developing a new model, based on the idea of a set of tokens, one for each member of the team, generated by the source, and distributed to neighbours with various possible strategies. Each node then counts tokens received from neighbours, using a network-wide collection algorithm, decides whether to keep one, effectively joining the team, and propagates the resulting number.

- The idea of relay agents may be eliminated, to test the impact of errors it induces.

# Bibliography


[1] Giorgio Audrito. Fcpp: an efficient and extensible field calculus framework. In *2020 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*, pages 153–159, 2020.

[2] Giorgio Audrito, Jacob Beal, Ferruccio Damiani, and Mirko Viroli. *Space-Time Universality of Field Calculus*, pages 1–20. 05 2018.

[3] Giorgio Audrito, Ferruccio Damiani, Mirko Viroli, and Roberto Casadei. Runtime management of computation domains in field calculus. In *2016 IEEE 1st International Workshops on Foundations and Applications of Self* Systems (FAS*W)*, pages 192–197, 2016.

[4] Giorgio Audrito and Marco Ottina. Aggregate drone monitoring of wildfires. In *2022 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCom/CyberSciTech)*, pages 1–6, 2022.

[5] Giorgio Audrito, Luigi Rapetta, and Gianluca Torta. Extensible 3d simulation of aggregated systems with fcpp. In *Coordination Models and Languages: 24th IFIP WG 6.1 International Conference, COORDINATION 2022, Held as Part of the 17th International Federated Conference on Distributed Computing Techniques, DisCoTec 2022, Lucca, Italy, June 13-17, 2022, Proceedings*, page 55–71, Berlin, Heidelberg, 2022. Springer-Verlag.

[6] Giorgio Audrito, Mirko Viroli, Ferruccio Damiani, Danilo Pianini, and Jacob Beal. A higher-order calculus of computational fields. *ACM Trans. Comput. Logic*, 20(1), jan 2019.

[7] Jacob Beal, Danilo Pianini, and Mirko Viroli. Aggregate programming for the internet of things. *Computer*, 48(9):22–30, 2015.

[8] Roberto Casadei, Danilo Pianini, and Mirko Viroli. Simulating large-scale aggregate mass with alchemist and scala. volume 8, 10 2016.

[9] European Commission. A europe fit for the digital age. Technical report.

[10] Matteo Francia, Danilo Pianini, Jacob Beal, and Mirko Viroli. Towards a foundational api for resilient distributed systems design. In *2017 IEEE 2nd International Workshops on Foundations and Applications of Self* Systems (FAS*W)*, pages 27–32, 2017.

[11] *FCPP Documentation.*

[12] *Asymptote official website.*

[13] Marco Mamei and Franco Zambonelli. Programming pervasive and mobile computing applications: The tota approach. *ACM Trans. Softw. Eng. Methodol.*, 18(4), jul 2009.

[14] Danilo Pianini, Mirko Viroli, and Jacob Beal. Protelis: Practical aggregate programming. In *Proceedings of the 30th Annual ACM Symposium on Applied Computing*, SAC '15, page 1846–1853, New York, NY, USA, 2015. Association for Computing Machinery.

[15] Mirko Viroli, Giorgio Audrito, Jacob Beal, Ferruccio Damiani, and Danilo Pianini. Engineering resilient collective adaptive systems by self-stabilisation. *ACM Trans. Model. Comput. Simul.*, 28(2), mar 2018.

[16] Mirko Viroli, Jacob Beal, Ferruccio Damiani, Giorgio Audrito, Roberto Casadei, and Danilo Pianini. From distributed coordination to field calculus and aggregate computing. *Journal of Logical and Algebraic Methods in Programming*, 109:100486, 2019.

[17] Mirko Viroli, Jacob Beal, Ferruccio Damiani, and Danilo Pianini. Efficient engineering of complex self-organising systems by self-stabilising fields. pages 81–90, 09 2015.

[18] Mirko Viroli, Ferruccio Damiani, and Jacob Beal. A calculus of computational fields. In Carlos Canal and Massimo Villari, editors, *Advances in Service-Oriented and Cloud Computing*, pages 114–128, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.